

สคริปต์นำเสนอฉบับสั้น

เป้าหมายเวลา 5-8 นาที · พูดคนเดียว ไม่มีกรรมการถาม

0:00 แนะนำตัว

0:20 ภาพรวม/โจทย์

1:45 Favorite

3:15 Rating

5:15 averageRating

6:30 ปิดท้าย

ทุกจุดที่มี  คือให้เปิด/ชี้ไฟล์-บรรทัดนั้นบนจอตอนพูดประโยคนั้น แนะนำเปิดไฟล์ทั้งหมดเป็นแท็บ VS Code เตรียมไว้ก่อนตามลำดับในหน้าสุดท้าย จะได้แค่คลิกสลับแท็บ ไม่ต้องเสิร์ชสๆ กล่องสีฟ้า "พูดว่า..." คือแนวพูด ไม่ต้องท่องจำเป๊ะ พูดด้วยคำตัวเองได้เลยจะเป็นธรรมชาติกว่า

1) แนะนำตัว

0:00 - 0:20

"สวัสดีครับ ผม **[ชื่อ-นามสกุล]** รหัสนักศึกษา **[รหัส]** วันนี้ขอนำเสนองานสอบกลางภาควิชา Golang ครับ"

แก้ชื่อ/รหัสนักศึกษาให้ตรงกับตัวเองก่อนพูดจริงนะครับ ส่วนนี้สั้นๆ ไม่ต้องยัดเยียด 10-15 วินาทีพอ

2) สรุปความเข้าใจของตัวเอง

0:20 - 1:45

Wongnok คืออะไร

"Wongnok คือโปรเจกต์ REST API สูตรอาหาร ที่ผมโคลนโครงสร้างเริ่มต้นมาจากของอาจารย์ครับ เขียนด้วยภาษา Go เป็น backend อย่างเดียว ยังไม่มีหน้าเว็บ ทดสอบผ่าน Swagger หรือ Postman"

Tech stack ที่ใช้ และแต่ละตัวทำหน้าที่อะไร

"เทคโนโลยีหลักมี 6 ตัวครับ: **Gin** เป็น framework สร้าง web server คล้ายๆ Express ของ Node.js, **GORM** ช่วยคุยกับฐานข้อมูลโดยไม่ต้องเขียน SQL ดิบทุกบรรทัด, **Postgres** คือฐานข้อมูลหลักเก็บสูตรอาหารกับผู้ใช้, **Redis** เก็บข้อมูลชั่วคราวตอนล็อกอิน, **Keycloak** เป็นระบบล็อกอินสำเร็จรูปแยกต่างหาก แอปเราไม่เก็บรหัสผ่านเอง และ **goose** ใช้จัดการการเปลี่ยนโครงสร้างฐานข้อมูลที่ละเอียด"

โจทย์อาจารย์ให้ทำอะไร

"โจทย์คือให้ต่อยอดจาก Recipe API เดิมที่มีแค่ รายการสูตร ดูสูตรเดียว และสร้างสูตร โดยเพิ่ม 2 ฟังก์ชันครับ: **หนึ่ง Favorite recipes** คือกดโปรด/เลิกโปรดสูตรอาหารได้ และกรองรายการสูตรด้วย query favorite=true หรือ false ได้ **สอง Rating** คือให้คะแนนสูตรอาหารได้ แล้วต้องมี field averageRating ส่งกลับมาด้วยทั้งตอนดูรายการและดูสูตรเดียว"

วันนี้จะพูดอะไรบ้าง

"ผมจะพาไปดูทีละฟังก์ชันที่เพิ่มเข้าไปครับ ว่าแต่ละอันทำงานยังไง แต่ละบรรทัดสำคัญ หมายถึงอะไร และข้อมูลไหลจาก request เข้ามาจนตอบกลับยังไง"

3) ฟังก์ชันที่เพิ่ม

1:45 – 6:30

3.1 Favorite — POST /recipes/:id/favorite

1:45 – 2:35

internal/recipe/handler.go — ฟังก์ชัน Favorite

"ฟังก์ชันนี้ทำงาน 3 ขั้นครับ: อ่าน userID จาก token ก่อน (ถ้าไม่ได้ล็อกอินตอบ 401 ทันที) แปลง id สูตรอาหารจาก URL เป็นตัวเลข แล้วส่งต่อให้ service.Favorite(...) ทำงานต่อ — service จะเช็คก่อนว่าสูตรนี้มีอยู่จริงไหม ถ้าไม่มีตอบ 404 ถ้ามีก็ส่งต่อให้ repository ไปเขียนลงฐานข้อมูลตาราง user_favorites "

repository.go บรรทัด 150-162 — จุดสำคัญที่สุดของฟังก์ชันนี้

```
Clauses(clause.OnConflict{
    Columns: []clause.Column{{Name: "user_id"}, {Name: "recipe_id"}},
    DoUpdates: clause.Assignments(map[string]interface{}{
        "deleted_at": nil, "updated_at": time.Now(),
    }),
}).Create(&UserFavorite{...})
```

"ถ้าเขียนแบบเพิ่มแถวใหม่ตรงๆ ทุกครั้ง พอมีคนกดโปรด-เลิก-กดใหม่ จะเจอปัญหาแถวซ้ำ เพราะระบบลบแบบ soft delete ไม่ได้ลบจริง ผมเลยตั้งฐานข้อมูลว่า 'ถ้าเจอแถวเดิมของคู่ user กับสูตรนี้อยู่แล้ว ให้ซับซ้อนแถวเดิมกลับมาแทนที่จะสร้างใหม่' ทำให้ไม่มีทางเกิดแถวซ้ำเลย"

3.2 Unfavorite (DELETE) + กรอง favorite=true/false

2:35 – 3:15

handler.go ฟังก์ชัน Unfavorite

"เลิกโปรดใช้ DELETE /recipes/:id/favorite โครงเหมือน Favorite ทุกอย่าง ต่างแค่ repository ใช้ Delete() ตรงๆ เพราะแค่เอาออก ไม่ต้องกังวลเรื่องซ้ำ"

dto.go บรรทัด 98 Favorite *bool — repository.go บรรทัด 103-110

"ส่วนตอนกรองรายการ ผมใช้ตัวแปรชนิดพิเศษ *bool ที่แยกได้ 3 สถานะ คือไม่ส่ง query มา / ส่ง true / ส่ง false ถ้าใช้ boolean ธรรมดาจะแยกไม่ออกระหว่าง 'ไม่ได้ตั้งใจกรอง' กับ 'ตั้งใจกรองเอาเฉพาะที่ยังไม่กด' แล้วผมใช้ subquery หา recipe_id ที่ user คนนี้เคยกดโปรดไว้ แล้วกรองด้วย IN หรือ NOT IN"

3.3 Rating — POST /recipes/:id/rating

3:15 – 5:15

handler.go ฟังก์ชัน Rate

"ให้คะแนนมี endpoint เดียว รับค่า 1-5 ให้ซ้ำได้ ค่าล่าสุดแทนที่ค่าเดิม flow เหมือน Favorite: handler เช็ค login → แปลง id → bind body JSON เข้า req.Score → ส่งต่อ service เช็คสูตรมีจริงไหม → ส่งต่อ repository ไปทำงานจริง"

"ฟังก์ชันนี้ทำ 3 ขั้นตอน ห่อไว้ใน transaction เดียวกัน: **หนึ่ง** บันทึกลงหรือแทนที่คะแนนของ user คนนี้ด้วย OnConflict แบบเดียวกับ Favorite ถ้าให้คะแนนซ้ำ เช่นเคยให้ 4 แล้วเปลี่ยนเป็น 2 ต้องแทนที่เป็น 2 ไม่ใช่เอา 4 กับ 2 มาเฉลี่ยกันเป็น 3 **สอง** คำนวณค่าเฉลี่ยใหม่จากคะแนนทุกคนของสูตรนี้ **สาม** เขียนค่าเฉลี่ยนั้นกลับไปเก็บที่ตัวสูตรอาหารเลย เพื่อให้ตอนอ่านไม่ต้องคำนวณใหม่ทุกครั้ง"

"เหตุผลที่มัดทั้ง 3 ขั้นตอนเป็น transaction เดียว คือถ้าแยกทำ แล้วบังเอิญมี 2 คนให้คะแนนพร้อมกันเป๊าะๆ อาจเกิดเคสคำนวณค่าเฉลี่ยไปก่อนที่คะแนนคนแรกจะบันทึกเสร็จ ทำให้ค่าเฉลี่ยผิดค้างอยู่ถาวร"

3.4 averageRating — คำนวณและแสดงผล 5:15 – 6:00

📁 repository.go บรรทัด 192-195

```
Select("COALESCE(AVG(score), 0)").
  Where("recipe_id = ? AND deleted_at IS NULL", recipeID).
  Scan(&avgScore)
```

"จุดคำนวณจริงคือ AVG(score) จากทุกคะแนนของสูตรนี้ ผมครอบด้วย COALESCE ไว้ เพราะถ้ายังไม่มีใครให้คะแนนเลย AVG() จะได้ค่าว่าง ไม่ใช่ 0 ซึ่งจะทำให้โปรแกรม error ตอนแปลงเป็นตัวเลข เลยสั่งให้เป็น 0 แทนไว้ก่อน"

📁 dto.go บรรทัด 151, 198

"ค่าที่คำนวณได้ถูกเก็บไว้ใน column average_rating ของตัวสูตรอาหารเลย พอมีคนเรียกดูสูตรอาหาร ไม่ว่าจะดูรายการหรือดูเดี่ยว ระบบก็แค่หิ้วค่านี้มาใส่ใน field averageRating ของ response ไม่ต้องคำนวณใหม่ ตรงตามที่โจทย์ต้องการเป๊ะครับ"

ปิดท้าย 6:00 – 6:30

"สรุปคือ ผมเพิ่มฟีเจอร์ Favorite กับ Rating ครบตามโจทย์ พร้อมจัดการเคสที่มักพลาดกัน เช่นการกดซ้ำ การให้คะแนนซ้ำ และการคำนวณค่าเฉลี่ยให้ถูกต้อง ขอขอบคุณครับ"

ถ้าซ่อมแล้วยังเกิน 8 นาที ติดตามลำดับนี้ก่อน:

1. ส่วนกรอง favorite=true/false (3.2 ครั้งหลัง) — พุดสั้นลงเหลือประโยคเดียวว่า "กรองได้ด้วย query favorite" ไม่ต้องพุดเรื่อง subquery
2. ส่วนอธิบาย COALESCE (3.4) — พุดสั้นลงเหลือประโยคเดียว ไม่ต้องขึ้นบรรทัดก็ได้
3. ส่วนเหตุผล transaction (3.3 ย่อหน้าที่ 2) — ตัดออกได้ทั้งย่อหน้าถ้าจำเป็นจริงๆ

ห้ามตัด (เป็นแก่นของฟีเจอร์ตามโจทย์): repository.go บรรทัด 150-162 (Favorite/OnConflict), repository.go บรรทัด 171-207 ทั้งฟังก์ชัน (Rate/averageRating)

สารบัญไฟล์ที่ต้องเปิดเตรียมไว้ (เรียงตามลำดับพุด)

#	ไฟล์ / บรรทัด	ใช้ตอนไหน
1	internal/recipe/handler.go	Favorite, Unfavorite, Rate (ฟังก์ชัน)

2	<code>internal/recipe/repository.go</code> 150-162	Favorite / OnConflict
3	<code>internal/recipe/dto.go</code> 93-99, 151	Favorite *bool, AverageRating field
4	<code>internal/recipe/repository.go</code> 92-138	List / subquery กรอง favorite
5	<code>internal/recipe/repository.go</code> 171-207	Rate — จุดหลักของคลิป
6	<code>internal/recipe/dto.go</code> 157-198	NewRecipeResponse — ใส่ averageRating